Comparison of Matrix and Tensor Decomposition in Separation of Video Background and Foreground

MATH 6310

Department of Mathematics, University of Texas at Arlington

Abstract:

The aim of this report is to determine if separation of video background from foreground is more effective when data is kept as a tensor in comparison to vectorizing the frames and storing them as a matrix. The matrix decomposition algorithms applied are: R-PCA, CUR and IRCUR while the tensor decomposition algorithms applied are CP Decomposition using ALS, HOOI and RTCUR. Results show only R-PCA produces clear separation. Tucker and CP show promising results on synthetic data but are limited in computational time. More time needs to be spent on implementation of the methods to evaluate their performance further.

Felix Sjöholm and Yug Bhavsar

Submission Date: 2025-11-20

Contents

1 Introduction

Due to the surge in size of data recently, there is a need for dimension reducing methods to work more effectively while retaining an acceptable accuracy. By reducing the dimension, the data can be denoised, saves storage space and provide computational savings. If the dimension is reduced to 2 or 3 it can also be visualized, with many possible applications stemming from the visualization.

An interesting analysis to work with is how dimension reducing algorithms aimed for tensors compare to dimension reducing algorithms aimed for matrices. Videos are known to be represented by tensors with 2 dimensions representing width and height of each frame, another dimension representing the time unfolding of the video and the final dimension representing the intensity of the colours. In our report we will work with grayscale videos so that the tensor is of order 3. By stacking the columns in each video frame on top of each other, the vectorized frames can be placed in a matrix. Consequently, we have a way of comparing dimension reduction methods on data when it is kept as a tensor and when it has been matricized.

Intuition tells us that when the data is stored as a tensor, spatial relations between nearby pixels is retained and can be utilized in the different methods whereas this structure is lost when the frames are vectorized. Therefore, the tensor methods should perform better, at least with regards to accuracy. Due to the simplicity of matrix methods, one can assume that they should perform faster but this is no guarantee as the column dimension of the matrix will be very large in comparison to the dimensions of the tensor.

1.1 Matrices

1.1.1 Introduction

In order to use matrix methods, the data needs to be vectorized. By stacking each column of a video frame into one, the video frame is vectorized and all vectors, each representing one video frame, can be stored as columns in a matrix. Since the video frames all have the same background, a reasonable assumption is to assume that the vectors all share a large similarity and thus the matrix can be assumed to be low-rank. The foreground can reasonably be assumed to be sparse and thus we can decompose our matrix X, containing the video frames as vectors, into:

$$X = L + S \tag{1.1}$$

where L is the low-rank approximation and S is the matrix containing sparse outliers.

1.1.2 Principal Component Analysis using augmented Lagrangian

Principal Component Analysis (PCA) can be used to achieve this separation. Information regarding PCA and methods of computing it in this section are taken from [4]. Principal Component Analysis aims to find the best l^2 rank-k approximation of X by solving:

$$\begin{aligned} &\text{minimize} && ||X - L||_2 \\ &\text{subject to} && \text{rank}(L) \leq k \end{aligned} \tag{1.2}$$

However, for this approximation to be made possible, the sparse matrix S that was used earlier needs to be replaced with N, a small perturbation matrix. As this is not a reasonable approximation for our data, the Robust version of PCA (RPCA) is more suitable, where the data X can be separated into a low-rank matrix L and a matrix S, where the values in S can have

arbitrarily large values and their support is assumed to be sparse. This problem can be solved using Principal Component Pursuit (PCP), which estimates solving:

$$\begin{aligned} &\text{minimize} && ||L||_* + \lambda||S||_1 \\ &\text{subject to} && X = L + S \end{aligned} \tag{1.3}$$

With the choice of $\lambda = \frac{1}{\sqrt{n_{(1)}}}, n_{(1)} = \max(n_1, n_2)$, where n_1 and n_2 are the dimensions of the data matrix, the solution will always return the correct answer. Proof for this method's success can be found in [4], where the sparse outliers are assumed to follow a Bernoulli distribution. For the purpose of recovering background in a video, this assumption is not completely accurate as the moving objects will tend to be spacially coherent and a model such as Markov random fields might be more appropriate. Thus, there can be no guarantee that the algorithm will succeed with high probability, but we can still safely assume that an acceptable approximation should be possible to find.

R-PCA Algorithm

The algorithm proposed in [4] builds upon the augmented Lagrangian:

$$l(L, S, Y) = ||L||_* + \lambda||S||_1 + \langle Y, X - L - S\rangle \frac{\mu}{2}||X - L - S||_F^2 \tag{1.4}$$

A generic algorithm solving this problem would aim to find the L and S that minimize the problem whereas this algorithm finds them separately. By defining the shrinkage operator as $\mathcal{S}_\tau[x] = \text{sgn}(x)\max(|x| - \tau, 0)$ and extending it to matrices, it can be shown that:

$$\arg\min_S l(L, S, Y) = \mathcal{S}_{\lambda\mu}(M - L + \mu^{-1}Y) \tag{1.5}$$

By also defining singular value threshholding operator as $\mathcal{D}_\tau(X) = U\mathcal{S}_\tau(\Sigma)V^T$, where $X = U\Sigma V^T$ denotes any SVD, it can be shown that:

$$\arg\min_L l(L, S, Y) = \mathcal{D}_\mu(M - S - \mu^{-1}Y) \tag{1.6}$$

A more practical solution is to first minimize l with respect to L (fixing S) before minimizing l with respect to S (fixing L) and finally updating the Lagrange multiplier matrix Y based on the residual $M - L - S$. The pseudocode of the algorithm is shown below:

Algorithm 1 (Principal Component Pursuit by Alternating Directions [32, 51])

1: **initialize:** $S_0 = Y_0 = 0, \mu > 0$.
2: **while** not converged **do**
3: compute $L_{k+1} = \mathcal{D}_\mu(M - S_k - \mu^{-1}Y_k)$;
4: compute $S_{k+1} = \mathcal{S}_{\lambda\mu}(M - L_{k+1} + \mu^{-1}Y_k)$;
5: compute $Y_{k+1} = Y_k + \mu(M - L_{k+1} - S_{k+1})$;
6: **end while**
7: **output:** L, S.

Figure 1.1: Pseudocode for the R-PCA algorithm, taken from [4].

A python implementation of specifically this method can be found on GitHub.

1.1.3 Principal Component Analysis using CUR

Another way of performing PCA is to use matrix CUR decomposition instead of PCP. Information in this section is taken from [3]. In this article, the non-convex optimization problem of

RPCA is defined as follows:

$$\begin{aligned} \min_{L,S} \quad & ||X - L - S||_F \\ \text{subject to} \quad & \text{rank}(L) \leq k \text{ and } ||S||_0 \leq \alpha n^2 \end{aligned} \tag{1.7}$$

where k is the rank of the underlying low-rank matrix and α is the sparsity level of S. As stated before, the method we will look at uses the CUR decomposition of a matrix to find the optimal solution. The CUR decomposition aims to select columns and rows to approximate the original matrix. The columns are gathered in a matrix C, the rows in a matrix R and the overlap of the rows and columns are defined to be the matrix U. If a sufficient amount of columns and rows have been sampled, the original matrix X can be found as $X = CU^{\dagger}R$, where $\dagger$ denotes the pseudoinverse of a matrix.

CUR Algorithm

The CUR decomposition can be used as an approximation itself and pseudocode for an algorithm using this decomposition is shown in figure 1.2. Using an inputed target rank k, k randomly selected columns and rows are sampled uniformly. C and R are formed using these sampled columns/rows while U is formed as the intersection of the sampled rows/columns. The pseudoinverse of U is computed and the approximation is calculated and returned. This method is the simplest and thus should not give the best results. It will however be very computationally efficient compared to the others that iterate until a target error or convergence is reached.

Pseudocode : CUR Matrix Decomposition

Input: Matrix $X \in \mathbb{R}^{m \times n}$, target rank k

Output: Low-rank approximation L

1. Sample k column indices J uniformly at random.
2. Sample k row indices I uniformly at random.
3. Form $C = X(:, J)$.
4. Form $R = X(I, :)$.
5. Form $U = X(I, J)$.
6. Compute the pseudoinverse $U^{\dagger}$.
7. Construct the approximation:

$$L = C\, U^{\dagger}\, R.$$

Figure 1.2: Pseudocode for the CUR Matrix Decomposition.

Iterated Robust CUR Algorithm (IRCUR)

Pseudocode for the algorithm is shown in figure 1.3. In other algorithms, the truncated SVD is used to update L, but for large matrices this can be very costly. Instead, CUR is used as a low rank approximator. Columns and rows of $X - S_{k+1}$ are sampled via indices $\mathcal{I}$ and $\mathcal{J}$ that have been uniformly sampled. From these sampled columns and rows, R_{k+1} and C_{k+1} are formed. $\mathcal{H}_r$ finds the best rank-r approximation (i.e. truncated SVD) of $X - S_{k+1}$, denoted as U_{k+1}. L_{k+1} can then be updated as $L_{k+1} = C_{k+1}U_{k+1}^{\dagger}R_{k+1}$ and is of rank r. This step is not actually calculated as we only need the CUR components of L_{k+1} to be saved. This part of the algorithm is called Phase 2. Phase 1 finds the update of S by hard thresholding. The operator is defined as:

$$[\text{HT}_{\zeta}X]_{i,j} = \begin{cases} X_{i,j}, & |X_{i,j}| > \zeta; \\ 0, & \text{otherwise.} \end{cases} \tag{1.8}$$

Results show that iteratively decaying thresholding values achieve sufficient results and so an initial thresholding value ζ_0 and a thresholding decay parameter γ is needed. These stages are iterated until the error is sufficiently small. The error is defined as:

$$e_k = \frac{||[X - L_k - S_k]_{\mathcal{I},:}||_F + ||[X - L_k - S_k]_{:,\mathcal{J}}||_F}{||D_{\mathcal{I},:}||_F + ||D_{:,\mathcal{J}}||_F} \tag{1.9}$$

Because the full decomposition is never calculated and only submatrices are being evaluated, the computational complexity per iteration is $\mathcal{O}(r^2 n \log^2(n))$.

Algorithm 1 **I**terated **R**obust **CUR** for RPCA (IRCUR)

Input: $\boldsymbol{D}$: observed corrupted data matrix; r: rank; ε: target precision level; ζ_0: initial thresholding value; γ: thresholding decay parameter; $|\mathcal{I}|, |\mathcal{J}|$: sampling number of rows and columns.
Uniformly sample row indices $\mathcal{I}$ and column indices $\mathcal{J}$.
$\boldsymbol{L}_0 = \boldsymbol{0}, \quad \boldsymbol{S}_0 = \boldsymbol{0}, \quad k = 0$
while $e_k > \varepsilon$ (e_k is defined as in (6)) **do**
    (Optional) Resample indices $\mathcal{I}$ and $\mathcal{J}$
    // Phase I: updating sparse component
    $\zeta_{k+1} = \gamma^k \zeta_0$
    $[\boldsymbol{S}_{k+1}]_{:,\mathcal{J}} = \mathcal{T}_{\zeta_{k+1}}[\boldsymbol{D} - \boldsymbol{L}_k]_{:,\mathcal{J}}$
    $[\boldsymbol{S}_{k+1}]_{\mathcal{I},:} = \mathcal{T}_{\zeta_{k+1}}[\boldsymbol{D} - \boldsymbol{L}_k]_{\mathcal{I},:}$
    // Phase II: updating low rank component
    $\boldsymbol{C}_{k+1} = [\boldsymbol{D} - \boldsymbol{S}_{k+1}]_{:,\mathcal{J}}$
    $\boldsymbol{U}_{k+1} = \mathcal{H}_r([\boldsymbol{D} - \boldsymbol{S}_{k+1}]_{\mathcal{I},\mathcal{J}})$
    $\boldsymbol{R}_{k+1} = [\boldsymbol{D} - \boldsymbol{S}_{k+1}]_{\mathcal{I},:}$
    $\boldsymbol{L}_{k+1} = \boldsymbol{C}_{k+1}\boldsymbol{U}_{k+1}^{\dagger}\boldsymbol{R}_{k+1}$ // Do not compute this step
    $k = k + 1$
end while
Output: $\boldsymbol{C}_k, \boldsymbol{U}_k, \boldsymbol{R}_k$: CUR decomposition of $\boldsymbol{L}$.

Figure 1.3: Pseudocode for the IRCUR algorithm, taken from [3].

1.2 Tensors

1.2.1 Introduction

Before discussing the decomposition methods for tensors, an introduction to tensors and operations on tensors is necessary. The following definitions follow from [6].

A formal definition is as follows: An N:th order tensor is an element of the tensor product of N vector spaces, each with its own coordinate system. They are denoted in literature as an italic capital letter, e.g. $\mathcal{X}$ and the order of a tensor is the number of dimensions, sometimes called ways or modes. A first order tensor is called a vector and a matrix is a second order tensor. Like rows and columns can be fixed in a matrix, a fiber is the result of fixing every index except one. Slices are the result of fixing all but two indices and result in a two-dimensional section of the tensor.

The norm of a tensor is defined as follows:

$$||\mathcal{X}|| = \sqrt{\sum_{i_1=1}^{I_1} \sum_{i_2=1}^{I_2} \cdots \sum_{i_N=1}^{I_N} x_{i_1 i_2 \ldots i_N}^2} \tag{1.10}$$

which is analogous to the Frobenius norm for matrices. Similar to the inner product for vectors,

the tensor inner product of two same-sized tensors $\mathcal{X}, \mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times ... \times I_N}$ is defined as follows:

$$\langle \mathcal{X}, \mathcal{Y} \rangle = \sum_{i_1=1}^{I_1} \sum_{i_2=1}^{I_2} ... \sum_{i_N=1}^{I_N} x_{i_1 i_2 ... i_N} y_{i_1 i_2 ... i_N} \tag{1.11}$$

Calculating tensor multiplication is complex and so we will only introduce the tensor n-mode product. With a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times ... \times I_N}$ and matrix $U \in \mathbb{R}^J \times I_N$, the tensor n-mode product is written as $\mathcal{X} \times_n U$ and is of size $I_1 \times ... \times I_{n-1} \times J \times I_{n+1} \times ... \times I_N$. The element-wise product is as follows:

$$(\mathcal{X} \times_n U)_{i_1 ... i_{n-1} j i_{n+1} ... i_N} = \sum_{i_n=1}^{I_N} u_{j i_n} \tag{1.12}$$

A tensor can be unfolded into a matrix along mode i by stacking every other mode into one. The mode-i unfolding of a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times ... \times I_n}$ is written as $\mathcal{X}_{(i)}$ and has dimension $I_i \times \prod_{I_i \neq I_j} I_j$.

Lastly, it is important to define the Khatri-Rao matrix product. For matrices $\mathbf{A} \in \mathbb{R}^{I \times K}$ and $\mathbf{B} \in \mathbb{R}^{J \times K}$, the Khatri-Rao product is:

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} \mathbf{a}_1 \otimes \mathbf{b}_1 & \mathbf{a}_2 \otimes \mathbf{b}_2 & ... & \mathbf{a}_K \otimes \mathbf{b}_K \end{bmatrix} \tag{1.13}$$

The resulting matrix is of size $(IJ) \times K$. The symbol $\otimes$ defines the Kronecker matrix product: for matrices $\mathbf{A} \in \mathbb{R}^{I \times J}$ and $\mathbf{B} \in \mathbb{R}^{K \times L}$, the Kronecker product is defined as:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} & ...a_{1J}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} & ... & a_{2J}\mathbf{B} \\ \vdots & \vdots & \ddots & \vdots \\ a_{I1}\mathbf{B} & a_{I2}\mathbf{B} & ... & a_{IJ}\mathbf{B} \end{bmatrix} = \begin{bmatrix} \mathbf{a}_1 \otimes \mathbf{b}_1 & \mathbf{a}_1 \otimes \mathbf{b}_2 & ... & \mathbf{a}_J \otimes \mathbf{b}_{L-1} & \mathbf{a}_J \otimes \mathbf{b}_L \end{bmatrix} \tag{1.14}$$

The resulting matrix is of size $(IK) \times (JL)$.

1.2.2 CANDECOMP/PARAFAC (CP) Decomposition

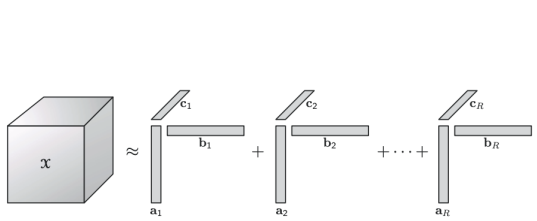
The CP decomposition factorizes a tensor into a sum of component rank-one tensors. The general CP decomposition for an N:th order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times ... \times I_N}$ is

$$\mathcal{X} \approx [\![\lambda; A^{(1)}, A^{(2)}, ..., A^{(N)}]\!] \equiv \sum_{r=1}^{N} \lambda_r \ \mathbf{a_r}^{(1)} \circ \mathbf{a_r}^{(2)} \circ ... \circ \ \mathbf{a_r}^{(N)} \tag{1.15}$$

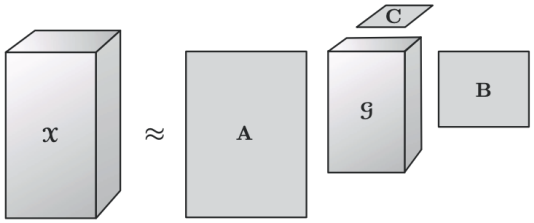
where λ_r is ... and $\circ$ represents the vector outer product:

$$\mathbf{u} \circ \mathbf{v} = \begin{bmatrix} u_1 v_1 & u_1 v_2 & ... & u_1 v_n \\ u_2 v_1 & u_2 v_2 & ... & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_n v_1 & u_n v_2 & \dots & u_n v_n \end{bmatrix} \tag{1.16}$$

Figure 1.4a is a visualization of how a third-order tensor can be CP decomposed. The rank of a tensor when discussing CP decompositions, denoted rank($\mathcal{X}$) is the smallest amount of rank-one tensors that sum to generate $\mathcal{X}$. The properties of a tensors rank varies from the properties of a matrices rank in that the rank can be different over $\mathbb{R}$ and $\mathbb{C}$. Also, determining a tensors rank is an NP-hard problem, with no specific algorithm/method.



(a) A 3:rd order CP decomposition of CP tensor rank R. The three modes are defined as a_i, b_i and c_i.



(b) A 3:rd order Tucker decomposition. $\mathcal{G}$ is the core tensor and A, B and C are the matrices of each mode.

Figure 1.4: Decomposition of a mode 3 tensor. Illustrations are from [6].

Many applications of CP Decompositions are discussed in [6], including its use in chemometrics, neuroscience, signal processing and even image compression. An advantage of CP decomposition is that it is computationally efficient [5] due to the simplicity of the decomposition. The decomposition, however, is not well suited in this research as the decomposition performed does not fit the data very well. There should be strong spatial correlations for pixels close together and in the time dimension the background should realistically be constant. Therefore, a method utilizing these characteristics, instead of assuming that the data consists of several rank-one tensors, should be more effective.

CP Decomposition using Alternating Least Squares

Pseudocode for this algorithm is shown in figure 1.5. The factor matrices that will be returned are randomly initialized. For each mode, the data tensor $\mathcal{X}$ is unfolded along the specified mode and the Khatri-Rao product is computed. The factor matrix is then updated and these steps are repeated until convergence is reached. The CP model is then returned.

Pseudocode : CP Decomposition using Alternating Least Squares (ALS)

Input: Tensor $X \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$, rank R
Output: Factor matrices $A^{(1)}, A^{(2)}, \ldots, A^{(N)}$

1. Initialize the factor matrices $A^{(n)}$ randomly for all modes $n = 1, \ldots, N$.
2. **Repeat until convergence:**
   (a) For each mode $n = 1, \ldots, N$:
      i. Unfold the tensor along mode n to form $X_{(n)}$.
      ii. Compute the Khatri–Rao product:

$$Z = A^{(N)} \odot \cdots \odot A^{(n+1)} \odot A^{(n-1)} \odot \cdots \odot A^{(1)}.$$

      iii. Update factor matrix:

$$A^{(n)} = X_{(n)} Z (Z^\top Z)^{-1}.$$

3. Return the CP model:

$$X \approx \sum_{r=1}^{R} \lambda_r \, a_r^{(1)} \circ a_r^{(2)} \circ \cdots \circ a_r^{(N)}.$$

Figure 1.5: Pseudocode for the CP-ALS Decomposition.

1.2.3 Tucker Decomposition

The Tucker decomposition is a form of tensor variant of PCA. The tensor is decomposed into a core tensor multiplied/transformed by matrices along each mode. Although it was introduced only for a mode-3 tensor, it can be defined for an N-way tensor as:

$$\mathcal{X} = \mathcal{G} \times_1 A^{(1)} \times_2 A^{(2)} ... \times_N A^{(N)} = [\![\mathcal{G}; A^{(1)}, A^{(2)}, ..., A^{(N)}]\!] \tag{1.17}$$

Figure 1.4b is a visualization of how a third-order tensor can be Tucker decomposed. When discussing Tucker decompositions, the n-rank, denoted $\text{rank}_n(\mathcal{X})$ of an N:th order tensor $\mathcal{X}$ of size $I_1 \times I_2 ... \times I_N$ is the column rank of $\mathcal{X}_{(n)}$. It can also be defined as the dimension of the column space spanned by the mode-n fibers. The Tucker rank can then be defined as the tuple $(r_1, ..., r_n) \in \mathbb{N}^n$, where $r_k = \text{rank}(\mathcal{X}_{(k)})$ [1].

To the Tucker decompositions disadvantage, a solution is not unique. Let $U \in \mathbb{R}^{P \times P}$, $V \in \mathbb{R}^{Q \times Q}$, $W \in \mathbb{R}^{R \times R}$ be nonsingular matrices. Then:

$$[\![\mathcal{G}; U, V, W]\!] = [\![\mathcal{G} \times_1 U \times_2 V \times_3 W; AU^{-1}, BV^{-1}, CW^{-1}]\!] \tag{1.18}$$

Equation 1.18 shows that the core $\mathcal{G}$ can be modified without affecting the decomposition as long as inverse modifications are applied to the factor matrices. For our research, we do not need the decomposition to be unique, as long as it is an accurate approximation of the original data. Therefore, this does not affect our choice of decomposition.

Tucker decomposition has been applied in fields such as chemometrics, signal processing, and, most importantly, image processing. Specifically, [6] raises the use of Tucker decomposition to retain key facial features while removing noise, such as lighting. Because the Tucker decomposition makes use of a core tensor, it is fitting to use this form of decomposition. The core tensor can capture interaction between different modes and since we expect nearby pixels in the spatial modes to have similar intensities, as well as the background to remain largely unchanged along the time mode, we feel that the Tucker decomposition is best for our research.

HOOI Algorithm

Under the assumption that $\mathcal{X}$ is of size $I_1 \times ... \times I_n$, the problem we wish to apply HOOI to can be formulated in the following way:

$$\begin{aligned} \min_{\mathcal{G}, A^{(1)}, ..., A^{(n)}} \quad & \|\mathcal{X} - [\![\mathcal{G}; A^{(1)}, ..., A^{(n)}]\!]\| \\ \text{subject to} \quad & \mathcal{G} \in \mathbb{R}^{R_1 \times ... \times R_n}, \\ & A^{(n)} \in \mathbb{R}^{I_n \times R_n} \text{ and columnwise orthogonal for } n = 1, ..., N \end{aligned} \tag{1.19}$$

Pseudocode to the algorithm is shown in figure 1.6. It builds upon the first known algorithm of computing the Tucker decomposition, namely the Higher-Order Singular Value Decomposition (HOSVD). It is called this because it is a convincing generalization of the matrix SVD [6]. $A^{(i)}$ is initialized by taking the SVD of the mode-i unfolding of $\mathcal{X}$. The leading R_i left singular vectors are chosen as $A^{(i)}$. Next, for each mode i a tensor $\mathcal{Y}$ is calculated as the l mode product of $\mathcal{X}$ with every $A^{(l)}, l \neq n$, and the leading R_i vectors of $Y_{(i)}$ are chosen as $A^{(i)}$. The previous step is repeated until the fit ceases to improve or when the maximum number of iterations has been reached. The core tensor $\mathcal{G}$ is calculated as the n-mode product of $\mathcal{X}$ with all $A^{(n)}$. This is returned along with the factor matrices $A^{(n)}$ along all modes. This method can be found in Tensorly's library.

procedure HOSVD($\mathcal{X}$,R_1,R_2,...,R_N)
  for $n = 1,\dots,N$ **do**
    $\mathbf{A}^{(n)} \leftarrow R_n$ leading left singular vectors of $\mathbf{X}_{(n)}$
  end for
  $\mathcal{G} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)\mathsf{T}} \times_2 \mathbf{A}^{(2)\mathsf{T}} \cdots \times_N \mathbf{A}^{(N)\mathsf{T}}$
  return $\mathcal{G}, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$
end procedure

(a) Pseudocode for calculating the Tucker Decomposition through HOSVD.

procedure HOOI($\mathcal{X}$,R_1,R_2,...,R_N)
  initialize $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$ for $n = 1,\dots,N$ using HOSVD
  repeat
    for $n = 1,\dots,N$ **do**
      $\mathcal{Y} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)\mathsf{T}} \cdots \times_{n-1} \mathbf{A}^{(n-1)\mathsf{T}} \times_{n+1} \mathbf{A}^{(n+1)\mathsf{T}} \cdots \times_N \mathbf{A}^{(N)\mathsf{T}}$
      $\mathbf{A}^{(n)} \leftarrow R_n$ leading left singular vectors of $\mathbf{Y}_{(n)}$
    end for
  until fit ceases to improve or maximum iterations exhausted
  $\mathcal{G} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)\mathsf{T}} \times_2 \mathbf{A}^{(2)\mathsf{T}} \cdots \times_N \mathbf{A}^{(N)\mathsf{T}}$
  return $\mathcal{G}, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$
end procedure

(b) Pseudocode for calculating the Tucker Decomposition through HOOI.

Figure 1.6: Pseudocode for calculating the Tucker Decomposition, taken from [6].

1.2.4 Tensor Fiber CUR Decomposition

The Tensor Fiber CUR Decomposition extends the Matrix CUR approach to tensors by sampling representative fibers along each mode and forming a compact core tensor that links them. All information regarding this decomposition is taken from [1]. For a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_n}$ with Tucker rank $(r_1, \dots r_n)$, the Fiber CUR Decomposition is defined as:

$$\mathcal{X} = \mathcal{R} \times_{i=1}^{n} (C_i U_i^{\dagger}) \tag{1.20}$$

With subsets $d_i \subseteq [I_i]$ and $p_i \subseteq [\prod_{I_i \neq I_j} I_j]$ we can define: $\mathcal{R} = \mathcal{X}(d_1, \dots, d_n), C_i = \mathcal{X}_{(i)}(:, J_i)$ and $U_i = C_i(I_i, :)$. Thus, C_i contains the selected fibers along mode i similarly to how C contains the chosen columns in in matrix CUR. $\mathcal{R}$ is a core tensor where all chosen fibers meet, similar to the matrix U in matrix CUR which connects the chosen rows and columns. U_i is chosen to be a matrix that links C_i and R_i. 1.7 shows a visual representation of a Tensor Fiber CUR Decomposition. The Computational complexity of computing this decomposition is dominated by the calculation of the pseudoinverse of U_i, which is of order $\mathcal{O}(r^2(n-1)log^2(d))$. Since we need to calculate U_i for all $i = 1, \dots, n$, this brings the order of complexity of the whole method to be $\mathcal{O}(r^2 n^2 log^2(d))$.

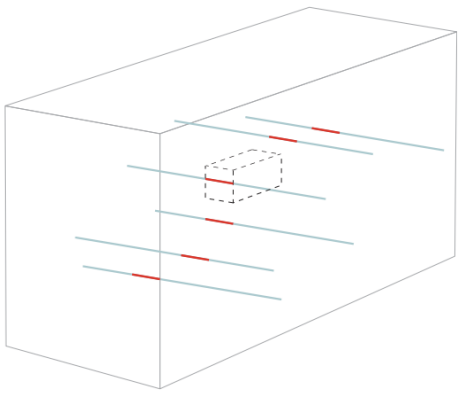


Figure 1.7: An illustration of the Fiber CUR decomposition. The lines correspond to chosen fibers in C_2, and red indices correspond to U_2. The outlined smaller tensor is R. Image is taken from [1].

Robust Tensor CUR (RTCUR) Algorithm

Pseudocode for the RTCUR algorithm is given in figure 1.8. The algorithm is an iterated process of projecting $\mathcal{X} - \mathcal{L}^{(k)}$ onto the space of sparse tensors, calling it $\mathcal{S}^{(k+1)}$. The less corrupted data $\mathcal{X} - \mathcal{S}^{(k+1)}$ is then projected onto the space of low-multilinear-rank tensors, calling it $\mathcal{L}^{(k+1)}$.

These are initialized to be the zero tensor. $\mathcal{S}$ is updated using a hard thresholding operator HT_ζ defined in equation 1.21:

$$(\mathrm{HT}_\zeta \mathcal{X})_{i_1,\ldots,i_n} = \begin{cases} \mathcal{X}_{i_1,\ldots i_n}, & |\mathcal{X}_{i_1,\ldots,i_n}| > \zeta; \\ 0, & \text{otherwise.} \end{cases} \tag{1.21}$$

Empirical findings show that iteratively decaying the threshold value ζ as $\zeta^{(k+1)} = \gamma\zeta^{(k)}$ provides superb performance with good choices for γ and $\zeta^{(0)}$. Next, $\mathcal{L}$ is updated by sampling the core tensor $\mathcal{R}$ from the chosen fibers $(I_1, \ldots I_n)$. $C_i^{(k+1)}$ is sampled by taking the $(J_1, \ldots J_n)$ columns of the mode-n unfolding of $\mathcal{X} - \mathcal{S}^{(k+1)}$. $U_i^{(k+1)}$ is found by taking the r_i truncated SVD of the $(I_1, \ldots I_n)$ sampled rows of $C_i^{(k+1)}$. $C_i^{(k+1)}$ and $U_i^{(k+1)}$ is found for all $i = 1, \ldots, n$. While $e^{(k)} > \varepsilon$, these steps are repeated. For time saving purposes, calculation of the Frobenius norm of the full tensor is avoided. Instead, the error is calculated as follows:

$$e^{(k)} = \frac{||\varepsilon^{(k)}(I_1, \ldots, I_n)||_F + \sum_{i=1}^n ||\varepsilon_{(i)}^{(k)}(:, J_i)||_F}{||\mathcal{X}(I_1, \ldots, I_n)||_F + \sum_{i=1}^n ||\mathcal{X}_{(i)}(:, J_i)||_F} \tag{1.22}$$

where $\varepsilon^{(k)} = \mathcal{X} - \mathcal{L}^{(k)} - \mathcal{S}^{(k)}$.

Input: $\mathcal{X} = \mathcal{L}_\star + \mathcal{S}_\star \in \mathbb{R}^{d_1 \times \cdots \times d_n}$: observed tensor; $(r_1, \cdots, r_n)$: underlying multilinear rank of $\mathcal{L}_\star$; ε: targeted precision; $\zeta^{(0)}, \gamma$: thresholding parameters; $\{|I_i|\}_{i=1}^n, \{|J_i|\}_{i=1}^n$: cardinalities for sample indices.
Initialization: $\mathcal{L}^{(0)} = \mathbf{0}, \mathcal{S}^{(0)} = \mathbf{0}, k = 0$
Uniformly sample the indices $\{I_i\}_{i=1}^n, \{J_i\}_{i=1}^n$
while $e^{(k)} > \varepsilon$ **do** // $e^{(k)}$ is defined in (14)
    (Optional) Resample the indices $\{I_i\}_{i=1}^n, \{J_i\}_{i=1}^n$
    // Step (I): Updating $\mathcal{S}$
    $\zeta^{(k+1)} = \gamma \cdot \zeta^{(k)}$
    $\mathcal{S}^{(k+1)} = \mathrm{HT}_{\zeta^{(k+1)}}(\mathcal{X} - \mathcal{L}^{(k)})$
    // Step (II): Updating $\mathcal{L}$
    $\mathcal{R}^{(k+1)} = (\mathcal{X} - \mathcal{S}^{(k+1)})(I_1, \cdots, I_n)$
    for $i = 1, \cdots, n$ **do**
        $C_i^{(k+1)} = (\mathcal{X} - \mathcal{S}^{(k+1)})_{(i)}(:, J_i)$
        $U_i^{(k+1)} = \mathrm{SVD}_{r_i}(C_i^{(k+1)}(I_i, :))$
    end for
    $\mathcal{L}^{(k+1)} = \mathcal{R}^{(k+1)} \times_{i=1}^n C_i^{(k+1)} \left(U_i^{(k+1)}\right)^\dagger$
    $k = k + 1$
end while
Output: $\mathcal{R}^{(k)}, C_i^{(k)}, U_i^{(k)}$ for $i = 1, \cdots, n$: the estimates of the tensor Fiber CUR decomposition of $\mathcal{L}_\star$.

Figure 1.8: Pseudocode for the RTCUR algorithm, which outputs the Fiber CUR decomposition of a tensor. Algorithm is taken from [1].

2 Problem Formulation

We will be applying dimension reducing algorithms to both synthetic data and grey-scale videos.

Synthetic data will be produced by randomly generating values ranging from 0 to 255 in a matrix of size $h \times w$. To this matrix, noise will be added with an N(0,ϵ) distribution, $\epsilon = 10^{-6}$. For each video frame, a different noise matrix will be added, resulting in 10 synthetic frames of video with noise that we want removed. We will create another synthetic video by adding a sparse matrix to each frame, with values ranging from [-500,500] randomly sampled into the matrix while ensuring sparsity. The grey-scale video was found online and has been downloaded as an mp.4 file for analysis. Both the synthetic data and gray-scale video will be referred to as "data".

Firstly, the data will be vectorized by stacking each column of each frame on top of each other, effectively vectorizing the individual frames, and placing them in a matrix of which will have size $h \cdot w \times 10$. The R-PCA, CUR and IRCUR algorithms, with pseudocode shown in figure 1.1, 1.2 and figure 1.3 respectively, will be applied to this matrix.

The tensor algorithms, HOOI with pseudocode in figure 1.6, CP with pseudocode in figure 1.5 and RTCUR algorithm with pseudocode in figure 1.8, will be applied to the data in tensor form.

For the synthetic data, we want to compare the methods computational time and accuracy of removing the noise/sparse outliers. The accuracy is measured using the Signal-To-Noise (SNR) ratio found in [2], defined below, where X is the synthetic frame without added noise or sparsity and L is the approximated frame:

$$\mathrm{SNR}_{\mathrm{dB}}(L) = 20\log_{10}\left(\frac{||X||_F}{||L-X||_F}\right) \tag{2.1}$$

For the grey-scale videos, we want to compare the visual separation of background and foreground of one video frame with the original frame.

Code for the R-PCA algorithm can be found here: `https://github.com/dganguli/robust-pca`. Code for the IRCUR method can be found here: `https://github.com/sverdoot/robust-pca`. Code for the HOOI algorithm can be found in TensorLy's library, with function name "Tucker". Code for the RTCUR algorithm can be found here: `https://github.com/huangl3/RTCUR`. It is written in MatLab so our implementation in Python, along with our implementation of all code above, can be found in A.

3 Results

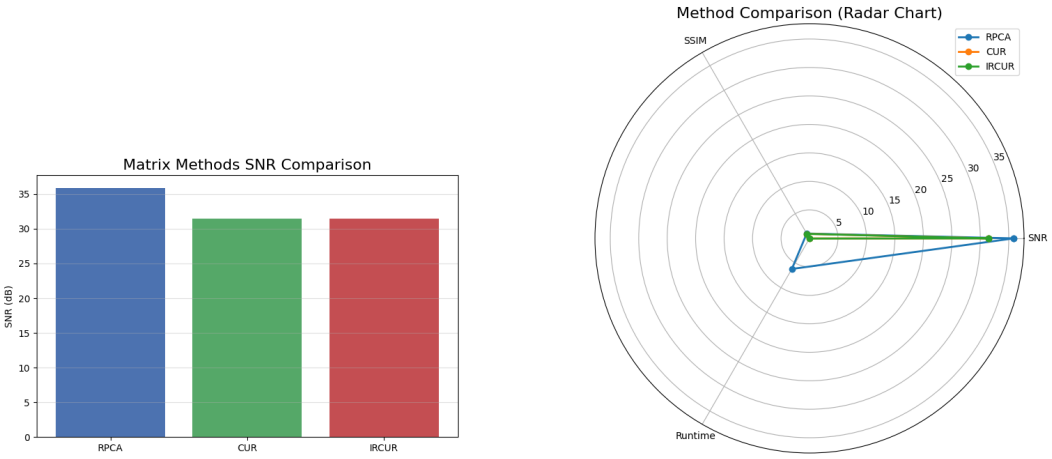
This section presents the quantitative and qualitative evaluation of matrix and tensor decomposition methods. We compare SNR, SSIM and runtime on synthetic data recovery and reconstruction quality on a greyscale video.

3.1 Matrix Decomposition Results

Matrix SNR (Synthetic):

- RPCA: 35.89 dB
- CUR: 31.44 dB
- IRCUR: 31.44 dB

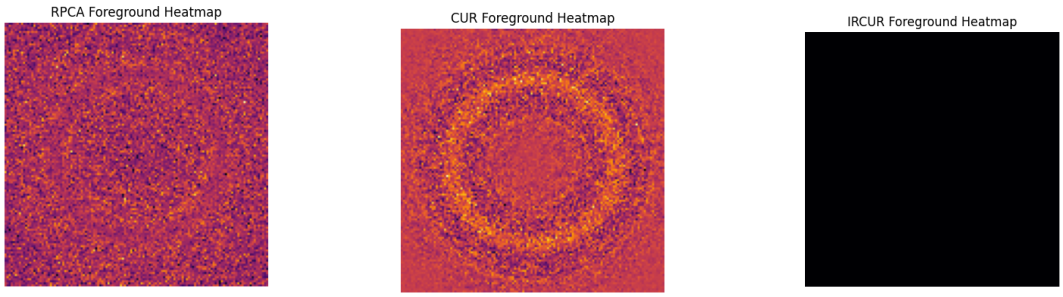
Figure 3.1 shows the SNR bar chart, using the values above, and radar chart (SNR–SSIM–Runtime) for the matrix methods. RPCA achieves the highest SNR and SSIM, while CUR and IRCUR provide faster but less accurate approximations.



(a) Matrix SNR Bar Chart (b) Matrix Radar Chart

Figure 3.1: Matrix method performance comparison.

Foreground intensity heatmaps are shown in figure 3.2 and reveal that RPCA produces the cleanest and most localized motion segmentation, whereas CUR and IRCUR introduce sampling artifacts.



(a) RPCA Foreground (b) CUR Foreground (c) IRCUR Foreground

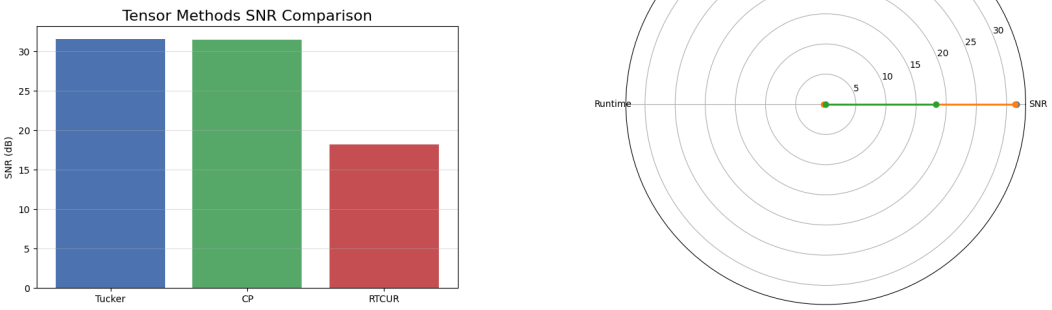
Figure 3.2: Foreground reconstruction heatmaps for matrix methods.

3.2 Tensor Decomposition Results

Tensor SNR (Synthetic):

- Tucker: 31.59 dB
- CP: 31.46 dB
- RTCUR: 18.22 dB

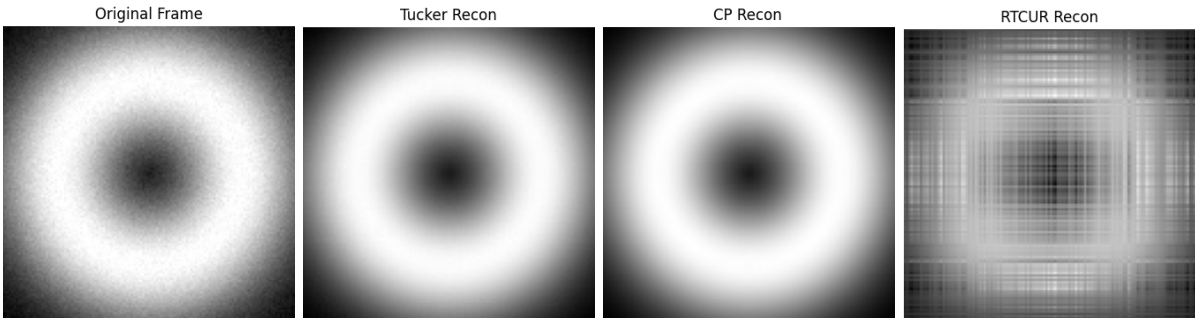
Figure 3.3 shows the tensor SNR comparison, using the values above, and radar chart (SNR–Runtime). Tucker and CP provide stable, high-quality reconstruction, while RTCUR shows a significant drop in SNR.



(a) Tensor SNR Bar Chart (b) Tensor Radar Chart

Figure 3.3: Tensor method performance comparison.

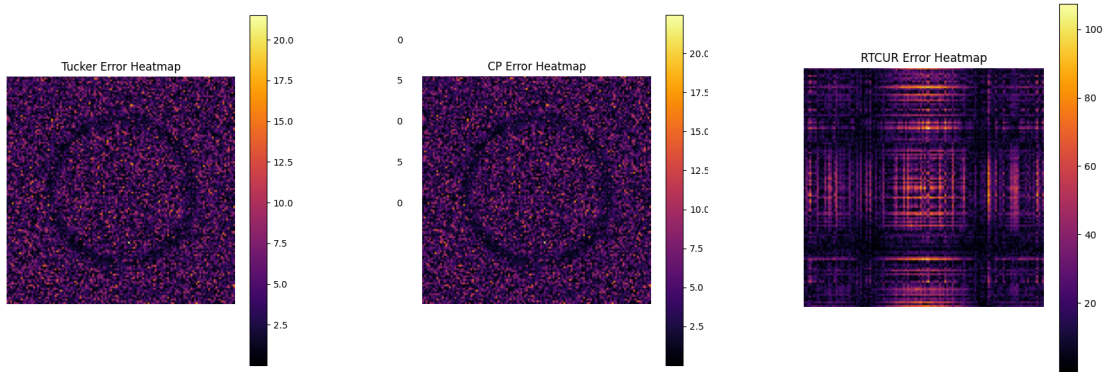
Reconstruction comparisons shown in figure 3.4 confirm that Tucker and CP preserve spatial structure well, whereas RTCUR exhibits visible distortions.



(a) Original (b) Tucker (c) CP (d) RTCUR

Figure 3.4: Tensor method reconstruction results.

Figure (Figure 3.5 show error heatmaps and highlight significantly higher error for RTCUR compared to Tucker and CP.



(a) Tucker Error (b) CP Error (c) RTCUR Error

Figure 3.5: Error heatmaps for tensor methods.

3.3 Video Background Separation

The real-video experiment results are shown in Figure 3.6. RPCA reconstructs the cleanest background and most accurate foreground. CUR and IRCUR detect motion but show sampling artifacts and degraded structure. Tucker and CP recover the background somewhat clearly but do not remove foreground. RTCUR once again exhibits visible distortions.

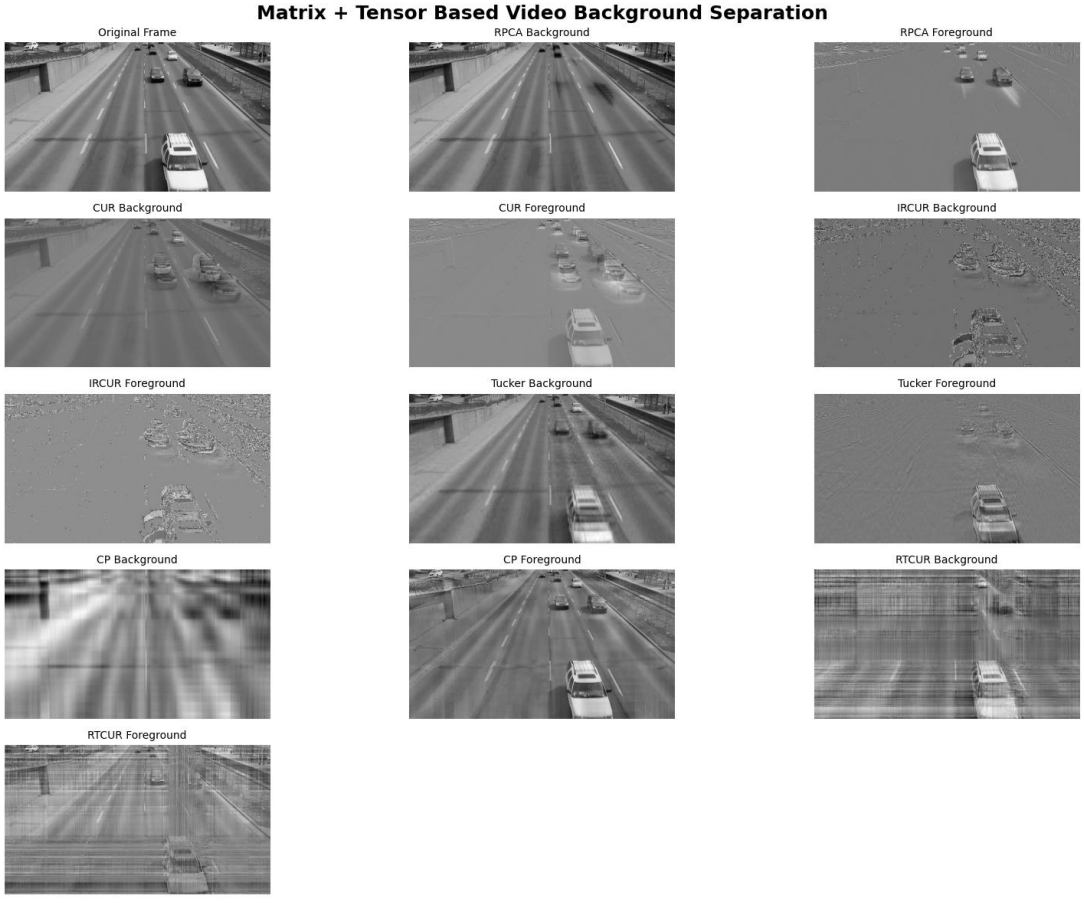


Figure 3.6: Video background–foreground separation using RPCA, CUR, IRCUR, Tucker, CP and RTCUR.

4 Discussion

This study compared six decomposition techniques for background–foreground separation in videos: the matrix-based RPCA, CUR, IRCUR and the tensor-based Tucker, CP, and RTCUR methods. The evaluation was performed on synthetic tensor data, which was matricized for the matrix methods, and real video frames. Metrics such as SNR, SSIM, runtime, and qualitative background–foreground separation were used to assess each method.

4.1 Matrix-Based Methods

RPCA produced the most accurate background reconstruction, achieving the highest SNR and SSIM among matrix methods. Additionally, the recovered background was smooth and closely matched the ground truth, and the sparse foreground effectively isolated moving objects. However, RPCA also required the longest runtime due to its iterative low-rank and sparse optimization.

CUR and IRCUR offered significantly faster runtime. Their performance is similar in both SNR and SSIM because both rely on sampling high-energy rows and columns. IRCUR improves robustness with iterative thresholding, but both methods still fall short of RPCA in reconstruction quality. Visible artifacts and reduced background smoothness were observed in their outputs.

4.2 Tensor-Based Methods

Tensor methods preserve spatial and temporal dimensions, allowing them to model global video structure more effectively than matrix methods. Tucker decomposition achieved the highest SNR in the tensor experiments, followed closely by CP. Both methods reconstructed the background accurately and showed stable behavior across frames.

RTCUR performed significantly worse, with an SNR of only 18.22 dB. Since RTCUR uses fiber sampling instead of full multi-linear decomposition, it is more sensitive to noise and temporal variation. The error heatmaps confirmed larger deviations, especially in dynamic regions.

4.3 Video Experiment Analysis

In the real video experiment, RPCA again demonstrated the most stable background estimation and cleanest foreground separation. CUR and IRCUR detected motion but introduced grainy textures and sampling noise. Although IRCUR was slightly more robust than CUR, both underperformed compared to RPCA.

Tensor methods were also applied to the full video frames but did not perform as expected and had a longer computational time than the matrix methods.

4.4 Overall Findings

The combined results highlight key trade-offs among all six methods:

- RPCA provides the most accurate background–foreground decomposition but is computationally expensive.
- CUR and IRCUR are significantly faster and suitable for large-scale or real-time scenarios, but with reduced precision.

- Tucker and CP decompositions capture spatial and temporal structure effectively and yield strong reconstruction accuracy.
- RTCUR, although efficient, is highly sensitive to noise and exhibits the lowest reconstruction quality.

Overall, RPCA and Tucker stand out as the most reliable methods when reconstruction quality is the priority, whereas CUR, IRCUR, and RTCUR are better suited for applications requiring faster, approximate solutions.

4.5 Evaluation of Implementation of Algorithms

In the article related to the RTCUR algorithm ([1]), there is careful consideration to how the sampling, normalization and reconstruction are implemented. We may not have been able to reproduce this in Python, as this was the algorithm written in Matlab that needed to be rewritten in Python. Also, the data used is of larger dimension, with height $240 - 480$, width $320 - 640$ and $400 - 1250$ number of frames. Our synthetic data was of size $120 \times 120 \times 10$. Possibly, the method was not able to utilize fibers effectively as they were too short in length, thus making the approximation inaccurate. Similarly, the dimensions may not have been large enough for patterns to be captured along each mode of the Tucker decomposition, thereby making the approximation poor. Regarding the CP decomposition, as was stated in the introduction this is not a reasonable approximation of the background/foreground separation we want to achieve and so we did expect this method to perform worse in comparison to the others.

The CUR decomposition is meant to be a simple, fast method and thus we did not expect it to produce the best results but rather excel in computational time. In comparison to the robust IRCUR algorithm, it should not produce a better background/foreground separation and yet it does. Therefore, we can speculate that our implementation of the IRCUR algorithm was not flawless. Its result on the video footage is not acceptable but in the article discussing the method ([3], clear separation of background and foreground is achieved. Only the RPCA algorithm produced results that were expected. The moving cars are clearly visible in the sparse matrix whereas the background is mostly clear of moving objects. Unfortunately, it cannot be compared with the other methods, possibly due to the reasons mentioned above.

4.6 Related works

It is always important to compare results with related works. [3] uses the IRCUR algorithm to provide video background separation. Results show that there is clear background and foreground separation. In [1], RTCUR is compared to, among other algorithms, IRCUR and both methods achieve visually appealing background/foreground separation but with RTCUR providing much faster results. That a tensor method is faster than a matrix mathod is interesting as our results showed the opposite. In [4], background separation is also achieved and is more precise compared to an alternating minimization algorithm.

Thus, according to these sources, all methods should be able to achieve background/foreground separation. Unfortunately, possibly due to reasons mentioned above, we were not able to recreate these results.

Bibliography

[1] HanQin Cai et al. *Fast Robust Tensor Principal Component Analysis via Fiber CUR Decomposition.* 2021. arXiv: 2108.10448 [cs.LG]. URL: https://arxiv.org/abs/2108.10448.

[2] HanQin Cai et al. "Matrix Completion With Cross-Concentrated Sampling: Bridging Uniform Sampling and CUR Sampling". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.8 (). ISSN: 1939-3539. DOI: 10.1109/tpami.2023.3261185. URL: http://dx.doi.org/10.1109/TPAMI.2023.3261185.

[3] HanQin Cai et al. "Rapid Robust Principal Component Analysis: CUR Accelerated Inexact Low Rank Estimation". In: *IEEE Signal Processing Letters* 28 (2021), pp. 116–120. ISSN: 1558-2361. DOI: 10.1109/lsp.2020.3044130. URL: http://dx.doi.org/10.1109/LSP.2020.3044130.

[4] Emmanuel J. Candes et al. *Robust Principal Component Analysis?* 2009. arXiv: 0912.3599 [cs.IT]. URL: https://arxiv.org/abs/0912.3599.

[5] Fiveable. *9.2 Tensor Decompositions (CP, Tucker) - Advanced Matrix Computations.* 2025. URL: https://fiveable.me/advanced-matrix-computations/unit-9/tensor-decompositions-cp-tucker/study-guide/qQk7yU50JIspwEHv (visited on 10/22/2025).

[6] Tamara G. Kolda and Brett W. Bader. "Tensor Decompositions and Applications". In: *SIAM Review* 51 (2009), pp. 455–500.

A Computer Code

Python code used in the report can be found at .